

8-Puzzle Solver using A* Algorithm

1. Abstract

This report presents the design and implementation of an A* search-based solution to the classical 8-puzzle problem. The study focuses on applying informed search techniques to efficiently determine an optimal sequence of moves from an initial configuration to a goal state. The project highlights the role of heuristic functions in improving search performance and reducing computational overhead.

2. Introduction

Artificial Intelligence provides a framework for solving complex problems through systematic search strategies. Among these, informed search algorithms incorporate domain-specific knowledge to guide exploration. The A* algorithm is a prominent example, widely recognised for its optimality and efficiency.

The 8-puzzle problem serves as a standard benchmark for evaluating search algorithms. It involves arranging tiles in a specific order by sliding them within a constrained grid.

3. Problem Statement

The objective of the 8-puzzle problem is to transform an initial configuration of tiles into a predefined goal configuration using the minimum number of moves. Each move consists of sliding a tile into the empty space.

4. Objectives

- To model the 8-puzzle problem as a state-space search problem
 - To implement the A* search algorithm
 - To apply a heuristic function to guide the search process
 - To obtain an optimal sequence of moves from initial to goal state
-

5. Literature Background

Search algorithms in Artificial Intelligence are broadly classified into uninformed and informed methods. Uninformed methods such as Breadth-First Search explore the state space without additional knowledge, often leading to higher computational costs. In contrast, informed methods such as A* utilise heuristics to prioritise promising paths.

6. Methodology

6.1 State Representation

The puzzle is represented as a one-dimensional list of nine elements. Each element corresponds to a tile, and the value 0 represents the blank space.

6.2 Successor Function

Successor states are generated by moving the blank tile in four possible directions, subject to boundary conditions.

6.3 A* Algorithm

The A* algorithm evaluates nodes based on the function:

$$f(n) = g(n) + h(n)$$

where: - $g(n)$ is the cost from the initial state to the current node - $h(n)$ is the estimated cost from the current node to the goal state

6.4 Heuristic Function

The misplaced tiles heuristic is used in this implementation. It calculates the number of tiles that are not in their goal position. This heuristic is admissible and guarantees optimality.

7. Implementation Details

The implementation is carried out in Python using standard libraries. A priority queue is employed to select the next node with the lowest evaluation function value. The algorithm maintains a visited set to avoid redundant exploration.

The system accepts user input as the initial configuration and outputs the sequence of steps required to reach the goal state.

8. Results and Discussion

The algorithm successfully determines an optimal solution for valid inputs. The use of the heuristic function reduces the number of explored states compared to uninformed search methods.

The output includes the total number of steps and the sequence of intermediate states, allowing clear interpretation of the solution path.

9. Advantages

- Guarantees optimal solution when using an admissible heuristic
 - More efficient than uninformed search algorithms
 - Demonstrates practical application of AI concepts
-

10. Limitations

- Memory consumption can be high for complex states
 - Performance depends on the quality of the heuristic function
-

11. Future Scope

- Implementation of Manhattan distance heuristic for improved performance
 - Development of a graphical interface for visualization
 - Extension to larger puzzles such as the 15-puzzle
 - Comparative analysis with other search algorithms
-

12. Conclusion

The project demonstrates the effectiveness of the A* algorithm in solving the 8-puzzle problem. It highlights the importance of heuristic functions in guiding search processes and achieving optimal solutions efficiently.

13. Learning Outcomes

- Understanding of informed search algorithms
- Practical implementation of A*
- Experience with heuristic evaluation
- Ability to model problems as state-space search